

6.1. Introducción / 6.2.7. Bloques: ¿Cuál es la diferencia fundamental entre el archivo como unidad lógica (la abstracción que ve el usuario) y cómo el sistema operativo manipula realmente la información en los dispositivos físicos de almacenamiento?

R: un archivo es la unidad lógica mínima de almacenamiento persistente en un sistema operativo. Al ser una estructura lógica, deben transformarse para que puedan ser almacenados en discos basados en bloques. Esta transformación se realiza mediante llamadas al sistema permitiendo la comunicación entre el usuario y el sistema operativo. El trabajo de lectura o escritura del sistema operativo con respecto a los archivos no es byte a byte, si se modifica algo en el archivo, este deberá leer el bloque completo en el que se encuentra el archivo hacia la memoria, modificarlo y devolverlo a disco.

6.2.1. y 6.2.2. Tabla de archivos abiertos: Cuando un archivo está siendo usado por múltiples procesos, ¿qué información crítica guarda el sistema operativo en la "Tabla global" y qué guarda específicamente en la tabla de cada proceso (descriptor)?

R: en sistemas multitarea tenemos que varios procesos pueden estar usando el mismo archivo. Podemos definir que puede hacer cada proceso sobre el archivo y como repercute a los demás que tienen el mismo archivo abierto. El sistema operativo registra la cantidad de procesos que emplean el archivo, los permisos de acceso que tiene el usuario a determinado recurso, la información para evitar que se tenga que consultar la tabla en disco para encontrar el archivo y los modos de acceso si se requiere acceder mutuamente al archivo. Por parte del proceso se guardan el descriptor de archivo, que es la relación entre el nombre del archivo con el identificador numérico que maneja el proceso, y los permisos que tiene el archivo para el proceso.

6.2.3. Bloqueo de archivos: Contrastá el bloqueo mandatorio (obligatorio) con el consultivo (advisory). ¿Qué pasa si un programa intenta modificar un archivo bajo un esquema de bloqueo puramente consultivo?

R: el bloqueo mandatorio consiste en que el encargado de imponer las restricciones de acceso a los demás procesos cuando uno toma el lock es el sistema operativo. Mientras que el bloque consultivo consiste en que es manejado cooperativamente entre todos los procesos y el programador es que debe manejar el bloqueo.

6.2.4. Tipos de archivo: Explicá las tres estrategias principales que utilizan los sistemas operativos (Windows, Unix y MacOS clásico) para reconocer el tipo y formato de un archivo.

R: los sistemas Windows para reconocer el tipo y formato del archivo utilizan extensiones, dividiendo el archivo en dos porciones separadas por un punto, el nombre y la extensión. El sistema guarda una lista de extensiones conocidas para saber que acción realizar ante una determinada extensión. Por su parte, los sistemas Unix utilizan números mágicos, guardan las huellas digitales de los formatos a manejar, para reconocer el contenido del archivo basado en sus

primeros bytes. Apple Macintosh dividió el archivo en dos, la información y los metadatos que constituyen al archivo.

6.2.5. y 6.2.6. Acceso y Archivos especiales: En sistemas Unix "todo es un archivo". ¿Qué representan los archivos de tipo "Dispositivo" y cómo identifican a qué hardware físico deben apuntar empleando los números mayor y menor?

R: los archivos de tipo dispositivo permiten el acceso a un dispositivo externo pudiendo ser de bloques o caracteres. El número menor apunta a la clase de dispositivo y el mayor a la instancia.

6.3.1. a 6.3.3. Estructuras de directorios: ¿Por qué el modelo de directorio como un grafo dirigido (usando enlaces duros y simbólicos) es mucho más flexible que un árbol estructurado rígido, pero a la vez presenta el riesgo de bucles al recorrerlos?

R: el directorio estructurado como un grafo dirigido permite que múltiples archivos o directorios se encuentren en un mismo nodo, así la multitarea es posible. Mientras que, en un árbol rígido, un directorio solo podía pertenecer a un mismo nodo, impidiendo que un archivo o subdirectorio se comparta, se debía garantizar que estos tengan nombres diferentes. Los bucles pueden surgir debido a los directorios especiales (.), (..) donde (.) es un enlace duro al directorio actual, (..) es un enlace al directorio padre, por lo que en un recorrido se debe notar la idea de que se puede repetir un directorio. Por ejemplo: Proyecto -> Pedro -> Miembro -> Proyecto. Y ahí un bucle sin fin.

6.3.6. Montaje de directorios: ¿Qué significa "montar" un sistema de archivos en Unix (con sus puntos de montaje) y cómo difiere radicalmente de la forma en que Windows (Mi Computadora) presenta los volúmenes al usuario?

R: montar un directorio consiste en ubicarlo en algún punto del árbol de archivos. En el sistema Unix los directorios son montados en un solo árbol. Este se fusiona al árbol ya estructurado ocultando lo que el directorio previamente tuviera. En el montaje en Windows cada volumen tiene un identificador, y se monta de manera automática en un sistema de directorio estructurado como un árbol de un solo nivel representando a los dispositivos del sistema.

6.4.1. y 6.4.2. FAT vs Unix: Compará los atributos básicos de FAT (oculto, solo lectura, sistema, archivado) con el esquema de permisos en 3 niveles (usuario, grupo, otros) que implementó el modelo tradicional de Unix.

R: los directorios FAT incluyen un byte de atributos para definir los permisos/características de los archivos:

- Oculto: el archivo no debe mostrarse al usuario si este está encendido.
- Solo lectura: debe impedir que las llamadas al sistema que requieran escritura se efectúen.
- Sistema: el archivo no debe ser fragmentado ni movido.
- Archivado: utilizado por herramientas de respaldo para identificar archivos modificados desde la última copia.

Por el lado de los sistemas Unix se asocio nueve bits para los permisos donde los primeros tres representan los permisos del usuario, los segundos tres para los permisos del grupo y los últimos para los otros.

6.4.3. Listas de control de acceso (ACL): Si bien las ACL otorgan un control de permisos mucho más fino, ¿qué desventajas introducen a nivel de almacenamiento (i-nodos) y de resolución de ambigüedades?

R: la lista de control de acceso puede traer dificultades de almacenamiento ya que estas deben guardarse en una estructura adicional junto con su i-nodo, es decir, el archivo. Esta estructura puede ser de tamaño variable por lo que los tiempos de acceso podrían ser mayores a medida que tengamos diferentes archivos con distintos permisos. La ambigüedad que puede surgir en este modelo para controlar permisos, es que un usuario puede pertenecer a dos grupos distintos con reglas contradictorias. Esto ultimo es resuelto por el sistema con las políticas preestablecidas.

6.5.1. NFS vs 6.5.3. AFS: ¿Qué diferencia conceptual clave existe entre la arquitectura cliente-servidor estricta del Network File System (NFS) y el esquema de "agnosticismo a la ubicación" del Andrew File System (AFS)? Además, ¿cómo maneja AFS la consistencia de los archivos cuando un usuario los edita?

R: NFS opera bajo el modelo cliente-servidor donde existían redes con algún servidor de archivos, y todas las estaciones tenían una instalación simple y acceso al entorno de usuario completo. Esta arquitectura era útil para enviar archivos grandes, pero en redes que sean confiables. En cambio, el sistema Andrew File System consistía en un modelo distribuido, por lo que los datos no están en un servidor particular, si no que los volúmenes pueden migrar entre nodos sin interrumpir el acceso. Las lecturas y escrituras se manejaban de la siguiente forma: un nodo que abría un archivo, se copia en el cliente y las escrituras/lecturas se realizan de manera local. Para que otro nodo pueda ver su información modificada, al cerrar el archivo se 'avisa' a los demás nodos para decidir si obtener el archivo modificado o su copia.

6.6.4. El volumen: ¿Qué función vital cumple el "superbloque" (o bloque de control de volumen) y por qué los sistemas de archivos modernos guardan copias de respaldo de este por todo el disco?

R: el sistema utiliza el superbloque para mantener datos sobre la estructura del sistema operativo, el tamaño de los bloques lógicos y su extensión, el sistema de archivos que utiliza. En caso de que se corrompan estos datos, el sistema guarda en el disco copias de esta así puede montar y utilizar el volumen,

6.6.5. Directorios y el i-nodo: A diferencia de FAT donde el directorio guarda toda la metadata, ¿qué es exactamente un i-nodo en sistemas modernos y qué relación tiene con el nombre del archivo?

R: un i-nodo es una estructura que almacena todos los metadatos del archivo e indica a que sectores del disco pertenece el archivo. Uno de los datos que

almacena el i-nodo es el link counter que indica cuantas referencias a ese archivo hay dentro de los directorios, se manejan mediante enlaces duros.

6.6.6. Compresión y Desduplicación: Diferenciá el concepto de "compresión archivo por archivo" del de "desduplicación". ¿Cómo utiliza la desduplicación los hashes criptográficos para ahorrar espacio en servidores grandes?

R: NO ENTIENDO NADA DE COMPRESIÓN Y DESDUPLICACIÓN.

6.7.1. Asignación contigua: ¿Por qué la asignación contigua ofrece la mejor velocidad de transferencia, pero sufre severamente de fragmentación externa al intentar agrandar un archivo?

R: en la asignación continua, la velocidad de transferencia es mejor que en otras asignaciones debido a que la información se encontraba en bloques adyacentes y el movimiento del cabezal era mínimo. Pero esto traía fragmentación externa debido a que, si un archivo se editaba, debía liberarse el lugar donde se ocupaba y ser movido a un hueco más grande.

6.7.2. Asignación ligada: En este esquema no hay fragmentación externa. Sin embargo, ¿qué sucede con la desalineación de los datos (potencias de 2) y por qué impone obligatoriamente un "acceso secuencial" a la información?

R: los clústeres necesarios para esta asignación se entregan completos, por lo que, si tenemos clústeres de 2048 y 4 bytes son para el apuntador al siguiente clúster, se entregan 2044 bytes rompiendo la alineación con las potencias de dos. El acceso secuencial corresponde a que, si queremos acceder a la información almacenada en un bloque dado, el sistema debe recorrer los bloques previos para llegar a su objetivo.

6.7.3. Asignación indexada: ¿Cómo resuelve el i-nodo (asignación indexada) los problemas de los métodos anteriores y cómo utiliza los "niveles de indirección" (sencilla, doble, triple) para representar archivos extremadamente grandes?

R: el i-nodo permite separar los apuntadores de los bloques de datos, disminuyendo la probabilidad de corrupción de los apuntadores. También permite guardar en caché los i-nodos que se estén ocupando en cierto momento, entonces los que no están siendo utilizados, no se necesita cargarlos. Las indirecciones son utilizadas cuando un archivo supera el nivel de almacenamiento de un i-nodo, por lo que se utiliza un bloque que contiene una lista adicional de apuntadores. En caso de requerir mucho más, podemos utilizar más indirecciones, pero estas traen consigo muchos accesos al disco.

6.8.1. y 6.8.2. Integridad: ¿Por qué operaciones simples como crear un archivo no son "atómicas" y qué trabajo intensivo debe realizar un programa como fsck para verificar la integridad de un disco tras un corte de energía?

R: las operaciones no son atómicas porque se necesitan realizar pasos intermedios para, por ejemplo, crear. Por lo que ante un fallo en estas operaciones intermedias el sistema de archivos podría quedar corrupto o perder

memoria. Un programa debe verificar la integridad de un disco revisando las estructuras que contienen los archivos.

6.8.3. Sistemas con bitácora (Journaling): Explicá qué es la bitácora (journal) y cómo este mecanismo evita tener que revisar todo el disco después de un apagón repentino.

R: la bitácora consiste en una estructura en la cual se almacena la 'historia' de las modificaciones de los metadatos. A medida que un archivo se actualiza, estos metadatos son agregados a la bitácora en caso de éxito. Si se produce un fallo y los cambios producen un error, solo basta ver el último caso de éxito en la bitácora y aplicar las restantes.

6.9.1. (TAR) a 6.9.4. (FAT): ¿Por qué el sistema de FAT (File Allocation Table) clásico presenta un alto riesgo de corrupción cruzada o cadenas perdidas si falla una escritura en medio de un proceso?

R: podría pasar que ante la eliminación de un archivo se genere un fallo provocando que este, si por ejemplo se marca como ignorado el apuntador al primer clúster relacionado al archivo (se produce el fallo), y no se 'limpió' los clústeres, se perderá memoria pues quedaron grabados los clústeres y un nuevo archivo que apunte a estos tendrá archivos cruzados.

6.9.5. UNIX y el concepto de Links: En el sistema de archivos de UNIX clásico, ¿cómo funciona el "contador de links" del i-nodo y por qué borrar un archivo (quitar su entrada del directorio) no siempre libera el espacio real en el disco inmediatamente?

R: los i-nodos llevan un contador de links debido a que en los sistemas Unix, por su la estructura de los directorios como grafos dirigidos, se pueden encontrar subdirectorios y archivos en diferentes directorios. Entonces, al borrar un archivo, el contador disminuye, si este es 0, no se direcciona más a los archivos y se puede liberar.

6.9.8. y 6.9.10. Mejoras modernas: ¿En qué consiste el mecanismo de "Copy on Write" a nivel de sistema de archivos (como en Btrfs o LogFS) y cómo esta técnica hace posible tomar snapshots (fotografías) instantáneas y seguras del sistema en vivo?

R: el Copy on Write a nivel de sistemas de archivos consiste en que luego de copiar los datos, en el lugar donde estos terminan realizar una copia de seguridad de la tabla de i-nodos, i-nodos, superblock, bitmap. Esto trae consigo que sea necesario el garbage collector para que vaya eliminando las viejas copias de segmentos no utilizados.